



DEVOPS OPERATING SYSTEM

OSOMUDEYA ZUDONU

SECTION 2.9: Networking Basics

Why Networking Knowledge Matters

You've built services. Now you need to troubleshoot them.

Common problems you'll face daily:

- **10:00 AM:** "I started the service but clients can't connect"
- **11:30 AM:** "Service won't start, says port already in use"
- **2:00 PM:** "Works on my machine but not in production"
- **4:00 PM:** "API returning 'connection refused'"

Every single one of these is a networking issue.

This section teaches you how to diagnose and fix them in minutes, not hours.

Part 1: Essential Networking Concepts

Concept 1: What Are Ports?

Ports are like apartment numbers for network services.

Think of your server's IP address as the building address, and ports as apartment numbers inside that building.

Visuals:

```
Your Server (IP: 54.123.45.67)
├── Port 22   → SSH (how you connect)
├── Port 80   → nginx (web traffic)
├── Port 8080 → payments-api (your app)
└── Port 5432 → PostgreSQL (database)
```

Why this matters:

- One IP address can host multiple services (you don't need separate servers)
- Each service listens on a different port (avoids conflicts)

- Clients connect to IP: PORT combination (e.g., 54.123.45.67:8080)

Example:

```
# These are different services on the same machine
http://54.123.45.67:80      # nginx
http://54.123.45.67:8080   # payments-api
```

What the system is thinking:

- "I have one IP address (54.123.45.67)."
- "But I need to run multiple services."
- "So I'll assign each service a different port number."
- "When traffic arrives, I check the port number to know which service to send it to."

Common mistakes beginners make: Trying to run two services on the same port (e.g., nginx and payments-api both on port 80).

Result: "Address already in use" error.

Solution: Use different ports (80 for nginx, 8080 for payments-api).

Concept 2: Localhost vs Network Addresses

There are two ways to reach a service:

Localhost (127.0.0.1):

- Only accessible from the same machine
- Used for testing
- Fast (no network involved)
- Bypasses firewalls (stays on the machine)

Example:

```
# You're on the server, testing locally
curl http://localhost:8080/health
# Works! (if service is running)
```

What the system is thinking:

- "localhost = 127.0.0.1 = this machine"
- "Traffic never leaves the server"
- "No firewall checks needed"

Network Address (Public IP):

- Accessible from other machines (your laptop, users, etc.)
- Used by real users
- Goes through firewalls (must be allowed)
- Can be blocked even if service is running

Example:

```
# Someone else trying to reach your server
curl http://54.123.45.67:8080/health
# Might fail if firewall blocks it
```

What the system is thinking:

- "Traffic comes from the internet"
- "Must pass through AWS Security Group (cloud firewall)"
- "Must pass through server firewall (ufw)"
- "If either blocks it, connection fails"

Why this matters:

- Localhost works ≠ Network works (firewall might block network access)
- Always test both when troubleshooting
- If localhost works but network doesn't: It's a firewall issue

Common mistakes beginners make: Testing only with `localhost` and assuming it works for everyone.

Reality: Users connect via the public IP, which might be blocked by firewalls.

Concept 3: What Does "Listening" Mean?

A service can be running but NOT listening on the network.

Service Status:

- Running = Process is active (`systemctl status` shows "active")
- Listening = Process is accepting network connections

Critical distinction:

Running $\neq$ Listening

What the system is thinking:

- "The process exists" (running)
- "But it's not accepting network connections" (not listening)

Example:

```
# Service is running
systemctl status payments-api
# Active: active (running) ←  Running

# But is it listening on port 8080?
sudo ss -tlnp | grep 8080
# (empty)  - Nothing listening!

# Problem: Service is running but not accepting connections
```

Why this happens:

1. Service crashed during startup (process exists but crashed)
2. Service is listening on the wrong port (check your config)
3. Service is bound to localhost only (127.0.0.1, not 0.0.0.0)
4. Port conflict (another service is using that port)

Common beginner mistake: Assuming "systemctl status shows active" means the service is accessible. Reality: It could be running but crashed, or listening on the wrong port.

How to check:

```
# Step 1: Is it running?
systemctl status payments-api

# Step 2: Is it listening?
sudo ss -tlnp | grep 8080

# If Step 1 = active but Step 2 = empty → Problem!
```

Concept 4: Firewalls Block Traffic

Even if the service is listening, firewalls can block connections.

The traffic path:

```
Your Laptop → Internet → AWS Security Group → Server Firewall → Service
               ✓           ✗ BLOCKED HERE           ✓           ✓
```

Two firewalls you'll deal with:

1. Server Firewall (ufw on Linux)

ufw (Uncomplicated Firewall) is the built-in firewall on Ubuntu/Debian that controls which ports are open on your server; it's already installed and you control it with the `ufw` command (e.g., `sudo ufw status`).

- Lives on your EC2 instance
- Blocks traffic at the OS level
- Even if AWS Security Group allows it, ufw can still block

2. Cloud Firewall (AWS Security Groups)

- Lives in AWS (before traffic reaches your server)
- Blocks traffic at the network level
- Even if ufw allows it, Security Group can still block

What the system is thinking:

- "Traffic arrives at AWS Security Group first"
- "If Security Group says DENY → Connection fails immediately"
- "If Security Group says ALLOW → Traffic reaches your server"
- "Then ufw checks: if ufw says DENY → Connection fails"
- "If ufw says ALLOW → Traffic reaches your service"

Why this matters:

- Both must say "ALLOW" or your connection fails
- Check both when troubleshooting
- Most common mistake: Opening one but forgetting the other

Common beginner mistake: "I opened port 8080 in AWS Security Group, why isn't it working?" Answer: Check ufw! It might be blocking it too.

The rule:

Both firewalls must allow traffic:

✓ AWS Security Group: Port 8080 ALLOW
✓ Server firewall (ufw): Port 8080 ALLOW
→ Connection works

✗ AWS Security Group: Port 8080 ALLOW
✗ Server firewall (ufw): Port 8080 DENY
→ Connection fails (blocked by ufw)

Part 2: Essential Diagnostic Tools

Tool 1: Check If Service Is Running

Command:

```
systemctl status nginx
```

What to look for:

- Active: active (running) ← Running
- Active: inactive (dead) ← Stopped
- Active: failed ← Crashed

Expected output:

```
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled;
   preset: enabled)
   Active: active (running) since Sat 2026-01-17 18:20:37 WAT; 4h 56min
   ago
     Docs: man:nginx(8)
   Process: 542 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on;
   master_process on; (code=exited, status=0/SUCCESS)
   Process: 561 ExecStart=/usr/sbin/nginx -g daemon on; master_process on;
   (code=exited, status=0/SUCCESS)
  Main PID: 599 (nginx)
    Tasks: 3 (limit: 1017)
   Memory: 3.4M (peak: 4.1M)
      CPU: 48ms
   CGroup: /system.slice/nginx.service
           └─599 "nginx: master process /usr/sbin/nginx -g daemon on;
   master_process on;"
             └─600 "nginx: worker process"
               └─601 "nginx: worker process"

Jan 17 18:20:36 ip-172-31-71-160 systemd[1]: Starting nginx.service - A
high performance web server and a reverse >
Jan 17 18:20:37 ip-172-31-71-160 systemd[1]: Started nginx.service - A high
performance web server and a reverse p>
```

What each part means:

- **Active:** active (running) = Service is running (this is what you want!)
- **Main PID:** 1234 = Process ID (useful for killing if needed)
- **enabled** = Service starts automatically on boot

If not running, start it:

```
sudo systemctl start nginx
```

Common beginner mistake: Only checking **systemctl status** and assuming the service is accessible. Reality: Running ≠ Listening. Always check both.

Tool 2: Check What Ports Are Listening

Use **ss** (socket statistics) - modern and fast:

```
# Show all listening TCP ports
sudo ss -tlnp
```

Expected output:

State	Recv-Q	Send-Q	Local Address:Port	Process
LISTEN	0	128	0.0.0.0:80	users:(("nginx",pid=1234))
LISTEN	0	128	0.0.0.0:8080	users:(("python3",pid=5678))
LISTEN	0	128	127.0.0.1:5432	users:(("postgres",pid=9012))

What each column means:

- **LISTEN** = Service is accepting connections (this is what you want!)
- **0.0.0.0:80** = Listening on all network interfaces, port 80
 - 0.0.0.0 means "accept connections from anywhere"
 - 127.0.0.1:80 would mean "only localhost" (not accessible from network)
- users:(("nginx",pid=1234)) = nginx process (PID 1234) owns this port

What to look for:

- Good- If you see your port listed → Service is listening (good!)
- Bad- If grep returns nothing → Service is NOT listening (problem!)

Check specific port:

```
# Check if port 8080 is listening
su
do ss -tlnp | grep :8080
```

Expected output (if listening):

```
LISTEN 0 128 0.0.0.0:8080 users:(("python3",pid=5678))
```

Expected output (if NOT listening):

```
(empty - no output)
```

Common variations:

```
# Check if specific port is listening
sudo ss -tlnp | grep :8080

# If you see output → Something is listening
# If empty → Nothing is listening on that port
```

Common beginner mistake: Forgetting **sudo** (won't show process names, only ports). Solution: Always use **sudo ss -tlnp** or **sudo netstat -tlnp**.

Tool 3: Find What's Using a Port

When you get "port already in use" error:

```
# Find what's using port 8080
sudo lsof -i :8080
```

Expected output:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
nginx	1234	root	6u	IPv4	12345	0t0	TCP	*:8080 (LISTEN)

What each column means:

- COMMAND = Process name (nginx)
- PID = Process ID (1234)
- USER = Owner (root)
- NAME = Port and state (*:8080 (LISTEN))

This tells you:

- nginx is using port 8080
- Process ID: 1234
- Owner: root

Now you can decide:

1. Stop nginx if you don't need it: `sudo systemctl stop nginx`
2. Change your app to use a different port
3. Kill the process if it's stuck: `sudo kill 1234`

Alternative command:

```
# Using fuser (another way to find port usage)
sudo fuser 8080/tcp
```

Expected output:

```
8080/tcp: 1234
```

Common beginner mistake: Killing processes without checking what they are first.
Solution: Always check with `lsof` or `fuser` before killing.

Tool 4: Test Connectivity

Use **curl** to test if you can reach a service:

Test Local Service:

```
curl http://localhost:8080/health
```

Success output:

```
{"status":"healthy"}
```

Failure output:

```
curl: (7) Failed to connect to localhost port 8080: Connection refused
```

What this means:

- Connection refused = Nothing is listening on that port
- Service might be stopped or listening on a different port

Test Remote Service:

```
# Test from your laptop (replace with your EC2 IP)
curl http://54.123.45.67:8080/health
```

Possible results:

1. **Success** - service is accessible:

```
{"status":"healthy"}
```

2. **Connection refused** - nothing listening on that port:

```
curl: (7) Failed to connect to 54.123.45.67 port 8080: Connection refused
```

What this means: Service is not running or not listening on that port.

3. **Timeout** - firewall is blocking:

```
curl: (28) Connection timed out
```

What this means: Firewall (AWS Security Group or ufw) is blocking the connection.

4. **Connection reset** - firewall dropped the packet:

```
curl: (56) Recv failure: Connection reset by peer
```

What this means: Firewall received the connection but immediately dropped it.

Common beginner mistake: Only testing with **localhost** and assuming network access works. Solution: Always test both localhost and public IP.

Tool 5: Check Port with Netcat

nc (netcat) tests if a port is open:

```
# Test if port 8080 is open
nc -zv localhost 8080
```

Success:

```
Connection to localhost 8080 port [tcp/*] succeeded!
```

Failure:

```
nc: connect to localhost port 8080 (tcp) failed: Connection refused
```

Why use **nc** instead of **curl**?

- **curl** needs HTTP service (won't work for databases, SSH, etc.)
- **nc** works with ANY port (HTTP, database, SSH, custom protocols)

Test remote port:

```
nc -zv 54.123.45.67 8080
```

Expected output (if open):

```
Connection to 54.123.45.67 8080 port [tcp/*] succeeded!
```

Expected output (if blocked by firewall):

```
nc: connect to 54.123.45.67 port 8080 (tcp) failed: Connection timed out
```

Common beginner mistake: Using **curl** to test non-HTTP services (databases, SSH, etc.). **Solution:** Use **nc** for any port, **curl** only for HTTP/HTTPS.

Tool 6: Check Firewall Status

On Your Linux Server:

```
# Check ufw firewall status  
sudo ufw status
```

Expected output (if inactive):

```
Status: inactive
```

Expected output (if active):

Status: active

To	Action	From
--	-----	----
22/tcp	ALLOW	Anywhere
80/tcp	ALLOW	Anywhere
8080/tcp	ALLOW	Anywhere

What to look for:

- Good -If you see your port in the list → Firewall allows it
- Bad - If your port is missing → Firewall might be blocking it
- **Status: inactive** = Firewall is off (allows everything)

Enable/disable firewall:

```
# Enable firewall
sudo ufw enable

# Disable firewall (not recommended in production)
sudo ufw disable
```

Allow specific port:

```
# Allow port 8080
sudo ufw allow 8080/tcp

# Check it was added
sudo ufw status
```

On AWS (Security Groups):

Steps:

1. Go to your **EC2 Console**
2. Click on your **instance**
3. Click "**Security**" tab

4. Check "Security groups"
5. Look at "Inbound rules"

What you should see:

Port	Protocol	Source
22	TCP	Your IP (or 0.0.0.0/0)
80	TCP	0.0.0.0/0
8080	TCP	0.0.0.0/0

What to look for:

- Good- If you see your port with **0.0.0.0/0** → Allows from anywhere
- Bad - If you see your port with specific IP → Only allows from that IP
- Bad - If your port is missing → Firewall is blocking it

Common beginner mistake: Opening port in AWS Security Group but forgetting ufw (or vice versa).

Solution: Check both! Both must allow traffic.

Part 3: The Troubleshooting Flowchart

Use this every time a service isn't accessible:

```
Service not working?
↓
Step 1: Is it running?
  Command: systemctl status SERVICE
  ↓ Yes
Step 2: Is it listening on a port?
  Command: sudo ss -tlnp | grep PORT
  ↓ Yes
Step 3: Can I reach it locally?
  Command: curl http://localhost:PORT
  ↓ Yes
Step 4: Can I reach it from my laptop?
  Command: curl http://PUBLIC_IP:PORT
  ↓ No
```

Step 5: Check server firewall

Command: `sudo ufw status`

↓

Step 6: Check AWS Security Group

AWS Console → EC2 → Security → Inbound rules

↓

Fix the blocking rule

Detailed Troubleshooting Steps

Step 1: Is it running?

```
systemctl status payments-api
```

What to look for:

Active: **active (running)** → Service is running

Active: **inactive (dead)** → Service is stopped (start it: `sudo systemctl start payments-api`)

Active: **failed** → Service crashed (check logs: `sudo journalctl -u payments-api -n 50`)

Common mistake: Forgetting to check if the service is actually running before checking ports.

Step 2: Is it listening on a port?

```
sudo ss -tlnp | grep 8080
```

What to look for:

- Good - If you see output → Service is listening (good!)
- Bad - If empty → Service is NOT listening (problem!)

Common mistake: Forgetting **sudo** (won't show process names).

If not listening, check:

- **Service logs:** `sudo journalctl -u payments-api -n 50`
- **Config file:** Is the port number correct?
- **Port conflict:** Is another service using that port? (`sudo lsof -i :8080`)

Step 3: Can I reach it locally?

```
curl http://localhost:8080/health
```

What to look for:

- Success response → Service works locally
- Connection refused → Service is not listening (even though it's running)

Common mistake: Assuming localhost works means network works (firewall might block network access).

If localhost fails:

- Service might be listening on wrong port
- Service might be bound to wrong interface (127.0.0.1 instead of 0.0.0.0)

Step 4: Can I reach it from my laptop?

```
curl http://Your IP:8080/health
```

What to look for:

- Success response → Everything works!
- Connection refused → Service not listening or wrong port
- Connection timed out → Firewall is blocking
- Connection reset → Firewall dropped the connection

Common mistake: Only testing localhost and assuming network access works.

If the network fails but localhost works:

- It's likely a firewall issue (go to Step 5 and 6)

Step 5: Check the server firewall

```
sudo ufw status
```

What to look for:

- If you see your port in the list → Firewall allows it
- If your port is missing → Firewall is blocking it

If blocked, allow it:

```
sudo ufw allow 8080/tcp  
sudo ufw reload
```

Common mistake: Opening port in AWS but forgetting ufw.

Step 6: Check AWS Security Group

Steps:

1. AWS Console → EC2 → Your instance
2. Security tab → Security groups
3. Inbound rules

What to look for:

- If you see your port → Firewall allows it
- If your port is missing → Firewall is blocking it

If blocked, add rule:

- Port: 8080
- Protocol: TCP

- Source: 0.0.0.0/0 (or specific IP for security)

Common mistake: Opening port in ufw but forgetting AWS Security Group.

Real Troubleshooting Session

Problem: "API not accessible from my laptop"

Step 1: Is it running?

```
systemctl status payments-api
```

Output:

```
Active: active (running) 
```

Result: Service is running.

Step 2: Is it listening on a port?

```
sudo ss -tlnp | grep 8080
```

Output:

```
(empty - no output) 
```

Result: Service is running but NOT listening!

Step 3: Check logs to find out why

```
sudo journalctl -u payments-api -n 50
```

Output:

```
ERROR: Port 8080 already in use
```

Result: Port conflict! Something else is using port 8080.

Step 4: Find what's using port 8080

```
sudo lsof -i :8080
```

Output:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
nginx	1234	root	6u	IPv4	12345	0t0	TCP	*:8080 (LISTEN)

Result: Old nginx process (PID 1234) is using port 8080.

Step 5: Kill the old process

```
sudo kill 1234
```

Result: Old process killed.

Step 6: Restart payments-api

```
sudo systemctl restart payments-api
```

Result: Service restarted.

Step 7: Verify it's listening

```
sudo ss -tlnp | grep 8080
```

Output:

```
LISTEN 0 128 0.0.0.0:8080 users:(("python3",pid=5678))
```

Result: Now listening!

Step 8: Test locally

```
curl http://localhost:8080/health
```

Output:

```
{"status":"healthy"}
```

Result: Localhost works!

Step 9: Test from laptop

```
curl http://54.123.45.67:8080/health
```

Output:

```
curl: (28) Connection timed out
```

Result: Firewall is blocking!

Step 10: Check server firewall

```
sudo ufw status
```

Output:

```
Status: active
To          Action    From
--          -
22/tcp      ALLOW     Anywhere
80/tcp      ALLOW     Anywhere
```

Result: Port 8080 is missing! Firewall is blocking it.

Step 11: Allow port 8080

```
sudo ufw allow 8080/tcp  
sudo ufw reload
```

Result: Firewall updated.

Step 12: Test from laptop again

```
curl http://54.123.45.67:8080/health
```

Output:

```
{"status": "healthy"}
```

Result: Success!

Total time: 3 minutes (vs. hours of guessing)

What we learned:

1. Service was running but not listening (port conflict)
2. Found the conflicting process with **ls****of**
3. Killed the old process
4. Restarted the service
5. Found firewall was blocking network access
6. Opened port in ufw
7. Everything works!

Quick Reference: Common Commands

Check service status:

```
systemctl status SERVICE_NAME
```

Check what's listening:

```
sudo ss -tlnp | grep :PORT
```

Find what's using a port:

```
sudo lsof -i :PORT
```

Test connectivity (local):

```
curl http://localhost:PORT
```

Test connectivity (remote):

```
curl http://PUBLIC_IP:PORT
```

Test any port:

```
nc -zv HOST PORT
```

Check server firewall:

```
sudo ufw status
```

Allow port in firewall:

```
sudo ufw allow PORT/tcp
```

Check service logs:

```
sudo journalctl -u SERVICE_NAME -n 50
```

What You Learned

Technical Skills:

- Understanding ports and how services use them
- Difference between running and listening
- Localhost vs network addresses
- How firewalls work (server and cloud)
- Using diagnostic tools (ss, lsof, curl, nc, ufw)
- Systematic troubleshooting process

Troubleshooting Process:

- Always check running → listening → localhost → network
- Use logs to find root cause
- Check both firewalls (server and cloud)
- Test incrementally (don't skip steps)

Real-World Context:

- Most "service not working" issues are networking problems
- Firewalls are the #1 cause of "works locally but not remotely"
- Port conflicts are common (always check with lsof)
- Systematic approach saves hours of guessing

Common Patterns:

- Running ≠ Listening (always check both)
- Localhost works ≠ Network works (check firewalls)
- One firewall allows ≠ Both allow (check both)
- Port conflict = "Address already in use" (use lsof to find it)

Part 4: Hands-On Tasks

TASK N1: Diagnose and Fix Port Conflict

Business Context

Ticket ID: DEVOPS-1701

Priority: High

Assignee: You

Situation: Trying to start a new service on port 8080, but getting "address already in use" error. Need to identify what's using the port and resolve the conflict.

Impact: New service can't start and its blocking deployment.

Your Mission

1. Identify what's using port 8080
2. Decide how to resolve the conflict
3. Start the new service successfully

Step-by-Step Guide

Step 1: Simulate the Problem

```
# payments-api is already running on port 8080
sudo systemctl status payments-api
# Should show: active (running)

# Try to start something else on port 8080
python3 -m http.server 8080

Expected error:
OSError: [Errno 98] Address already in use
```

Press **Ctrl+C** to exit

Step 2: Find What's Using Port 8080

Method 1: Use lsof

```
sudo lsof -i :8080
```

Expected output (two possible formats):

Format 1 - Shows port number:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
python3	1234	ubuntu	3u	IPv4	12345	0t0	TCP	*:8080 (LISTEN)

Format 2 - Shows service name:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
python3	6545	ubuntu	3u	IPv4	25235	0t0	TCP	*:http-alt (LISTEN)

Why the difference:

- Port 8080 is commonly known as "http-alt" in the system services file
- **lsof** shows the service name when it's in **/etc/services**
- Both formats mean the same thing: something is using port 8080
- **http-alt** = port 8080 (they're the same)

What this tells you:

- Process name: python3
- Process ID (PID): 6545 (or 1234) - this number will be different each time, use YOUR actual PID
- User: ubuntu
- Port: 8080 (shown as "http-alt" or "8080" - same thing)
- LISTEN = accepting connections

Important: Your PID will be different (6545 in your case, not 1234). Always use the actual PID from YOUR output.

Method 2: Use ss (alternative)

```
sudo ss -tlnp | grep :8080
```

Expected output:

```
LISTEN 0 128 0.0.0.0:8080 0.0.0.0:*  
users:(("python3",pid=6545,fd=3))
```

Note: Your PID will be different (6545 in your case). Always use the actual PID from YOUR output.

Common mistake: Forgetting **sudo** (won't show process names).

Step 3: Identify the Process

```
# Get details about the process (replace 6545 with YOUR actual PID  
from Step 2)  
ps -fp 6545
```

Expected output:

```
UID    PID  PPID  C  STIME TTY   TIME      CMD  
ubuntu 6545 5678 0 10:00 pts/0 00:00:01 /usr/bin/python3  
/opt/payments-api/api.py
```

Note: Your PID (6545) and command path will be different. Use the PID from YOUR **lsuf** output.

What to look for:

- **CMD column:** Shows what it is (/opt/payments-api/api.py = payments-api service)

- **PPID:** Parent Process ID (if PPID is 1, it's a systemd service)

Check if it's a systemd service:

```
systemctl status payments-api
```

If you see "Active: active (running)" → It's a systemd service (use `systemctl stop`)

Step 4: Resolve the Conflict

You have 3 options:

Option A: Stop the Service (Recommended for Systemd Services)

When to use: It's a systemd service, you're replacing it with a new version.

```
sudo systemctl stop payments-api
```

Verify it stopped:

```
sudo systemctl status payments-api  
# Should show: Active: inactive (dead)
```

Why this is best: Graceful shutdown, safe, can restart easily.

Option B: Change Port for New Service

When to use: You need both services running, old service is critical.

```
# Start new service on port 8081 instead  
python3 -m http.server 8081
```

Why this is useful: No downtime, safe for production.

Option C: Kill the Process (Last Resort)

When to use: NOT a systemd service, process is stuck, `systemctl stop` didn't work.

WARNING: Only if it's NOT a systemd service!

```
# Replace 6545 with YOUR actual PID from Step 2
sudo kill 6545
```

If that doesn't work:

```
sudo kill -9 6545 # Force kill (last resort)
```

Why this is last resort: No graceful shutdown, might lose data.

Common mistake: Using `kill` on systemd services (they auto-restart). Always check with `systemctl status` first.

Step 5: Verify Resolution

Check port is free:

```
sudo ss -tlnp | grep :8080
# Should be empty (no output)
```

Start your new service:

```
python3 -m http.server 8080 &
```

Verify it's listening:

```
sudo ss -tlnp | grep :8080
```

Expected output:

```
LISTEN 0 128 0.0.0.0:8080 0.0.0.0:*
```

```
users:(("python3",pid=9999,fd=3))
```

What to look for:

- **SUCCESS:** If you see LISTEN with NEW PID (different from your original PID, e.g., 9999 instead of 6545) → New service is running
- **FAILED:** If you see the same PID (e.g., still 6545) → Old service still running

Test it works:

```
curl http://localhost:8080
```

Expected output: HTML directory listing (service is responding)

```
Clean up test service:  
# Find PID  
ps aux | grep "http.server 8080" | grep -v grep  
  
# Kill it (replace with actual PID)  
kill <PID>
```

Common Beginner Mistakes

1. **Killing without checking what it is** → Might kill production service
 - **Solution:** Always use `ps -fp PID` first
2. **Using kill on systemd services** → Service auto-restarts, conflict returns
 - **Solution:** Use `systemctl stop` for systemd services
3. **Forgetting sudo with lsof** → Won't show process names
 - **Solution:** Always use `sudo lsof -i :PORT`
4. **Not verifying it worked** → Think it's fixed but conflict still exists
 - **Solution:** Always check with `ss -tlnp | grep :PORT` after fixing

Acceptance Criteria

Check all boxes:

- ☐ Reproduced the "address already in use" error
- ☐ Found what's using port 8080 (lsof or ss)
- ☐ Found the process ID (PID)
- ☐ Identified what the process is (ps)
- ☐ Determined if it's a systemd service
- ☐ Chose appropriate resolution method
- ☐ Successfully resolved the conflict
- ☐ Verified new service started and is listening
- ☐ Tested the new service works
- ☐ Can explain when to use each resolution option

What You Learned

Technical Skills:

- Using `lsof` and `ss` to find port conflicts
- Understanding process details with `ps`
- Knowing when to use `systemctl stop` vs `kill`
- Multiple resolution strategies

Troubleshooting Process:

- Find → Identify → Decide → Fix → Verify
- Always check what process is before killing
- Verify resolution worked before moving on

Real-World Context:

- Port conflicts are common during deployments
- `systemctl stop` is safer than `kill` (graceful shutdown)
- Changing ports is often safer than stopping production services
- Always verify after fixing

In Summary: You learned to diagnose port conflicts systematically: find what's using the port, identify what it is, choose the right fix, and verify it worked. This is critical for deployments and troubleshooting.

Acceptance Criteria

Check all boxes:

- ☐ Identified what was using port 8080
- ☐ Found the process ID (PID)
- ☐ Understood what the process was
- ☐ Resolved the conflict (chose an option)
- ☐ Verified new service started successfully
- ☐ Can explain when to use each resolution option

TASK N2: Fix Blocked Connection (Firewall)

Business Context

Ticket ID: DEVOPS-1735

Priority: High

Assignee: You

Situation: The payments api works fine when tested locally (curl localhost:8080), but customers can't reach it from the internet. Need to diagnose if the firewall is blocking traffic.

Impact: Service is running but inaccessible to users.

Your Mission

1. Verify service is running and listening
2. Test local connectivity (should work)
3. Test remote connectivity (should fail)
4. Fix AWS Security Group to allow traffic
5. Verify customers can now access it

Step-by-Step Guide

Step 1: Verify Service is Running

```
# Check service status  
sudo systemctl status payments-api
```

Expected: active (running)

Step 2: Verify Service is Listening

```
# Check if listening on port 8080  
sudo ss -tlnp | grep :8080
```

Expected output:

```
LISTEN 0 128 0.0.0.0:8080 0.0.0.0:* users:(("python3",pid=1234))
```

Service is listening on all interfaces (0.0.0.0)

Step 3: Test Local Connectivity

```
# Test from the server itself
curl http://localhost:8080/health

Expected:
{"status":"healthy","service":"payments-api","version":"1.0.0"}
```

Local access works!

Step 4: Test Remote Connectivity

From your laptop (not the Ec2 server), try:

```
# Replace with your EC2 public IP
curl http://<Your EC2 IP>:8080/health

Expected error:
curl: (28) Failed to connect to 54.123.45.67 port 8080: Connection
timed out
```

Remote access fails!

Why timeout instead of "connection refused"?

- Timeout = Firewall is dropping packets silently
- Connection refused = Nothing listening on port

Step 5: Check Server Firewall On Ec2

```
# Check ufw status
sudo ufw status
```

Expected:

Status: inactive

The server firewall is not the problem.

Step 6: Check AWS Security Group

Go to AWS Console:

1. Navigate to EC2 Dashboard
2. Click "Instances" in left sidebar
3. Click on your instance (devops-lab-vm)
4. Click "Security" tab
5. Under "Security groups", click the security group name (e.g., devops-lab-sg)
6. Click "Inbound rules" tab

Current rules you should see:

Security group rule ID	Type	Protocol	Port range	Source
sgr-xxxxx	SSH	TCP	22	<Your IP>/32
sgr-xxxxx	HTTP	TCP	80	0.0.0.0/0
sgr-xxxxx	HTTPS	TCP	443	0.0.0.0/0

No rule for port 8080!

Step 7: Add Security Group Rule

Still in AWS Console:

1. Click the "Edit inbound rules" button
2. Click "Add rule."
3. Configure the rule:
 - **Type:** Custom TCP
 - **Protocol:** TCP
 - **Port range:** 8080
 - **Source:** 0.0.0.0/0 (anywhere)
 - **Description:** payments-api service
4. Click "Save rules."

New rules should show:

Security group rule ID	Type	Protocol	Port range	Source
sgr-xxxxx	SSH	TCP	22	<Your IP>/32
sgr-xxxxx	HTTP	TCP	80	0.0.0.0/0
sgr-xxxxx	HTTPS	TCP	443	0.0.0.0/0
sgr-xxxxx	Custom TCP	TCP	8080	0.0.0.0/0 ← NEW

Step 8: Test Remote Connectivity Again

From your laptop:

```
# Replace with your EC2 public IP
curl http://54.123.45.67:8080/health
```

Expected:

```
{"status":"healthy","service":"payments-api","version":"1.0.0"}
```

It works! Remote access successful!

Also test in browser:

```
http://<your ip>:8080/health
```

Should see the JSON response.

Understanding Security Groups

AWS Security Groups are like a firewall in front of your server:

```
Internet User → Security Group → Your Server
                ↑
                Rules define what gets through
```

Default behavior:

- **Inbound:** All traffic BLOCKED unless explicitly allowed
- **Outbound:** All traffic ALLOWED

This is why your service worked locally but not remotely:

- Local traffic never goes through Security Group
- Remote traffic must pass Security Group rules

Acceptance Criteria

Check all boxes:

- ☐ Verified service is running
- ☐ Verified service is listening on port 8080
- ☐ Tested local connectivity (worked)
- ☐ Tested remote connectivity (failed)
- ☐ Identified missing Security Group rule
- ☐ Added rule for port 8080
- ☐ Verified remote access now works
- ☐ Understand difference between local and remote access

TASK N3: End to End Connection Troubleshooting

Business Context

Ticket ID: DEVOPS-1789

Priority: Critical

Assignee: You

Situation: Production alert, customers reporting "can't access the website." You need to diagnose the issue quickly using systematic troubleshooting.

Impact: Revenue loss, customer complaints, SLA breach.

Your Mission

Use the troubleshooting flowchart to diagnose and fix a mystery connection issue.

Step by Step Guide

Step 1: Let's simulate a realistic failure

```
# Stop nginx
sudo systemctl stop nginx

# Start a dummy process on port 80 (this will cause issues)
sudo python3 -c "import socket; s=socket.socket();
s.bind(('127.0.0.1', 80)); s.listen(1); input('Press Enter to
stop...')" &

# Note the PID
echo $!
```

Now port 80 has a process that ONLY listens on localhost (127.0.0.1), not the network (0.0.0.0).

Step 2: Start Troubleshooting

Customer report: "Website won't load at http://YOUR_IP"

Let's follow the flowchart:

Step 2.1: Is nginx running?

```
systemctl status nginx
```

Expected:

Active: inactive (dead)

Nginx is stopped!

Try to start it:

```
sudo systemctl start nginx
```

Expected error:

Job **for** nginx.service failed because the control process exited with error code.

Check logs:

```
sudo journalctl -u nginx -n 20
```

Expected error:

nginx: [emerg] **bind()** to 0.0.0.0:80 **failed** (98: Address already in use)

Aha! Port 80 is in use.

Step 2.2: What's using port 80?

```
sudo lsof -i :80
```

Expected output:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
python3 (LISTEN)	5678	root	3u	IPv4	12345	0t0	TCP	localhost:80

Found it! Python process on port 80.

Step 2.3: Is this process supposed to be here?

```
# Get full command  
ps -fp 5678 (replace with actual PID)
```

Expected: Shows the dummy socket command we ran

This is **NOT** a legitimate service. It's blocking nginx.

Step 3: Fix the Issue

```
# Kill the rogue process (replace with actual PID)  
sudo kill 5678  
  
# Verify port 80 is free  
sudo lsof -i :80  
# (should be empty now)
```

Start nginx:

```
sudo systemctl start nginx
```

Verify it's running:

```
systemctl status nginx
```

Expected: active (running)

Step 4: Verify Service is Listening

```
sudo ss -tlnp | grep :80
```

Expected output:

```
LISTEN 0 128 0.0.0.0:80 0.0.0.0:* users:(("nginx",pid=1234))
```

nginx is now listening on all interfaces (0.0.0.0)

Step 5: Test Local Connectivity

```
curl http://localhost
```

Expected Output: HTML from nginx welcome page

Step 6: Test Remote Connectivity

From your laptop:

```
# Replace with your EC2 public IP
```

```
curl http://54.123.45.67
```

Expected: HTML from nginx welcome page

Or open in browser:

```
http://54.123.45.67
```

Should see "Welcome to nginx!"

Step 7: Document the Incident

```
cat > ~/devops-tasks/logs/incident-nginx-port-conflict.txt << 'EOF'
INCIDENT REPORT: nginx Service Unavailable
=====
```

Date: 2025-12-04

Duration: ~15 minutes

Severity: Critical (customer-facing service down)

CUSTOMER REPORT:

"Website won't load at http://IP_ADDRESS"

TROUBLESHOOTING STEPS:

1. Checked nginx status

Command: `systemctl status nginx`

Result: inactive (dead) - service not running

2. Attempted to start nginx

Command: `sudo systemctl start nginx`

Result: Failed with "Address already in use"

3. Checked logs

Command: `sudo journalctl -u nginx -n 20`

Finding: Port 80 already in use

4. Identified process using port 80

Command: `sudo lsof -i :80`

Finding: Rogue python3 process (PID 5678)

5. Investigated process

Command: `ps -fp 5678`

Finding: Unauthorized test script left running

ROOT CAUSE:

A test script was left running on port 80, preventing nginx from binding to the port. When nginx was restarted (or server rebooted), it could not start due to port conflict.

RESOLUTION:

1. Killed rogue process (PID 5678)
2. Verified port 80 was free
3. Started nginx successfully
4. Verified local and remote connectivity

VERIFICATION:

- nginx status: active (running)
- Listening on port 80: confirmed
- Local test (curl localhost): success
- Remote test (curl PUBLIC_IP): success
- Customer access restored

TIMELINE:

- 14:00 - Customer report received
- 14:02 - Started troubleshooting
- 14:05 - Identified port conflict
- 14:07 - Killed rogue process
- 14:08 - nginx restarted successfully
- 14:10 - Verified customer access restored
- 14:15 - Incident documented

LESSONS LEARNED:

- Always clean up test processes
- Implement monitoring for service status
- Add alert for nginx down
- Document all troubleshooting steps

PREVENTION MEASURES:

- Add process cleanup to deployment scripts
- Implement automatic service monitoring
- Create runbook for port conflict scenarios
- Train team on proper testing procedures

STATUS:  RESOLVED

EOF

```
cat ~/devops-tasks/logs/incident-nginx-port-conflict.txt
```

Quick Reference: Common Commands

Check Service Status

```
systemctl status SERVICE_NAME
```

Check What's Listening

```
# Show all listening ports
```

```
sudo ss -tlnp
```

```
# Check specific port
```

```
sudo ss -tlnp | grep :PORT
```

```
# Alternative (older tool)
```

```
sudo netstat -tlnp
```

Find What's Using a Port

```
# Find process using port
```

```
sudo lsof -i :PORT
```

```
# Get process details
```

```
ps -fp PID
```

Test Connectivity

```
# Test local service
```

```
curl http://localhost:PORT
```

```
# Test remote service
```

```
curl http://IP:PORT
```

```
# Test any port (works for non-HTTP services)
```

```
nc -zv HOST PORT
```

```
# Test with timeout (5 seconds)
```

```
curl -m 5 http://HOST:PORT
```

Check Firewalls

```
# Server firewall (Linux)
sudo ufw status

# Allow port in server firewall
sudo ufw allow PORT/tcp

# AWS Security Group: AWS Console → EC2 → Instance → Security tab

Common Ports
22      SSH
80      HTTP
443     HTTPS
3306    MySQL
5432    PostgreSQL
6379    Redis
8080    Common app port
8443    Common HTTPS alternative
```

Troubleshooting Flow

```
# 1. Is it running?
systemctl status SERVICE

# 2. Is it listening?
sudo ss -tlnp | grep :PORT

# 3. Test locally
curl http://localhost:PORT

# 4. Test remotely
curl http://PUBLIC_IP:PORT

# 5. Check server firewall
sudo ufw status

# 6. Check AWS Security Group (AWS Console)
```

Port Conflict Resolution

```
# Find what's using port
sudo lsof -i :PORT

# Stop systemd service
sudo systemctl stop SERVICE

# Kill process (if not systemd service)
sudo kill PID

# Force kill (last resort)
sudo kill -9 PID
```

Common Error Messages

"Address already in use"	→ Port conflict (use lsof to find it)
"Connection refused"	→ Nothing listening on that port
"Connection timed out"	→ Firewall blocking (check ufw and AWS)
"No route to host"	→ Network/routing issue

Check Service Logs

```
sudo journalctl -u SERVICE_NAME -n 50
```

Troubleshooting Patterns Summary

Pattern 1: Service Won't Start

```
Error: "Address already in use"
↓
Find: sudo lsof -i :PORT
↓
Decide: Stop service, kill process, or change port
↓
Verify: systemctl status + ss -tlnp
```

Pattern 2: Can't Connect Remotely

```
Symptom: Local works, remote fails
↓
Check: sudo ufw status (server firewall)
↓
Check: AWS Security Group (cloud firewall)
↓
Fix: Add inbound rule for port
↓
Verify: curl from laptop
```

Skills You've Mastered

Diagnostic Process:

- Systematic troubleshooting (not random guessing)
- Following the flowchart
- Reading error messages
- Identifying root cause

Network Troubleshooting:

- Port conflict resolution
- Firewall configuration
- Local vs remote connectivity testing
- Security Group management

Production Skills:

- Incident response workflow
- Documentation practices
- Verification testing
- Pattern recognition

Portfolio Deliverables

```
~/devops-tasks/logs/
```

```
|— incident-nginx-port-conflict.txt    # Real troubleshooting
documentation
|— [Previous incidents from other sections]
```

You can show in interviews:

- "I diagnosed and resolved network connectivity issues"
- "I troubleshoot port conflicts using `lsof` and `ss`"
- "I configured AWS Security Groups for service access"
- "I followed systematic troubleshooting to resolve production incidents"

Interview Talking Points

Common interview questions you can now answer:

Q: "How would you troubleshoot a service that won't start?"

A: "First check if it's a port conflict with `lsof -i :PORT`, check logs with `journalctl`, identify the blocking process, and resolve appropriately, either stopping the conflicting service or changing the port."

Q: "Service works locally but customers can't access it. How do you debug?"

A: "I'd verify the service is listening on all interfaces with `ss -tlnp`, test local connectivity with `curl`, then check both server firewall and cloud security groups for missing rules."

Q: "What's the difference between 'connection refused' and 'timeout'?"

A: "Connection refused means nothing is listening on that port. Timeout usually indicates a firewall is dropping packets. Different causes require different fixes."